Agentes Conversacionais

UNIDADE 05

Construindo chatbots com o RASA

Com a ascensão do mercado de agentes conversacionais, aumenta também a ***demanda por ferramentas que facilitem o desenvolvimento*** *e mantenimento de tais sistemas. Nesta Unidade, você será apresentado ao* ***RASA****, uma plataforma de desenvolvimento de chatbots em Python, que oferece acesso a recursos avançados de inteligência artificial.*

Visão geral

O RASA é um *framework open-source* que automatiza vários processos de *machine learning*, visando ao desenvolvimento de *chatbots*. Ele tem funcionalidades que cobrem o entendimento das mensagens, gerenciamento de diálogo, geração de conversas e integração com outros canais de comunicação e APIs.

Uma das grandes vantagens na utilização do RASA é o **alto grau de customização** que podemos fazer em seu *pipeline*, além de contar com um *background* de Processamento de Linguagem Natural (PLN) e *Machine Learning* (ML), oferecido por bibliotecas já conhecidas, como o spaCy e *scikit-learn*.

SAIBA MAIS

O RASA tem uma comunidade muito ativa e documentação bem escrita, incluindo um canal no YouTube, *blog* com postagens regulares e exemplos prontos de aplicação em sua página no GitHub. Você pode verificar os casos de uso neste *link*.

Para utilizar o RASA, é preciso um ambiente Python rodando, com a biblioteca do RASA instalada, bem como suas dependências. Neste caso, devido a algumas particularidades do RASA, a utilização do Google Colab traria algumas restrições. Por isso, vamos utilizar um ambiente local.

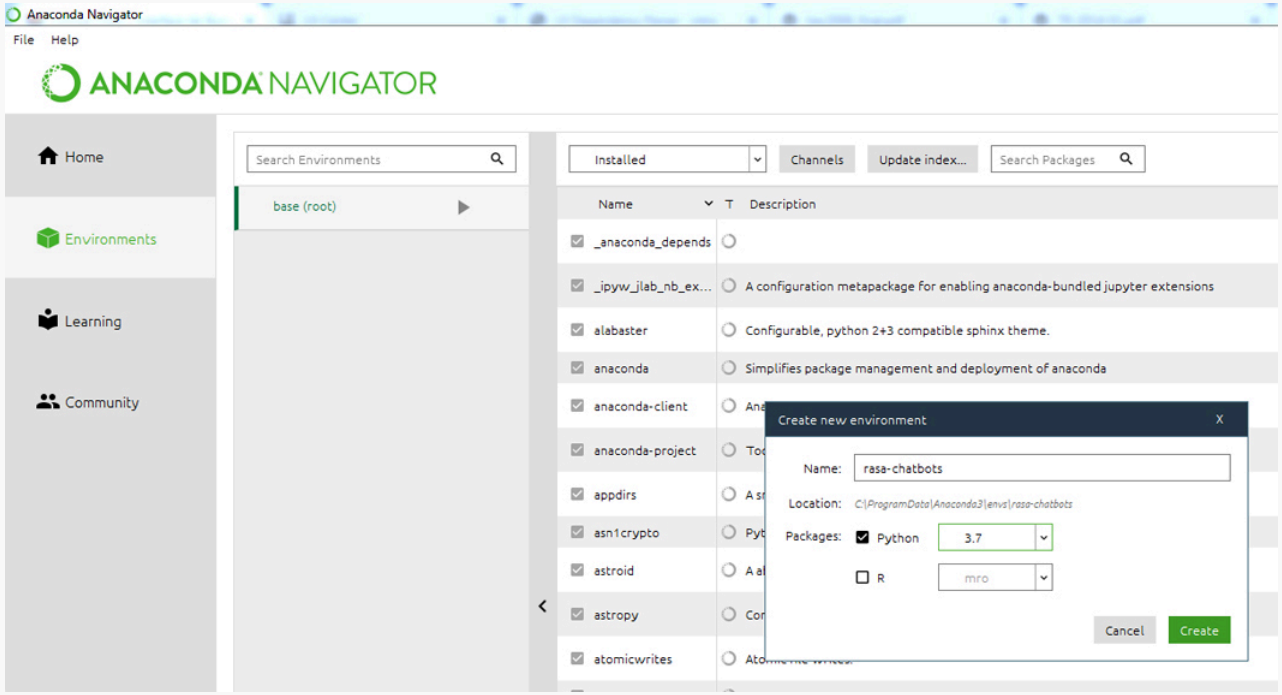
Você pode optar por instalar diretamente o Python no seu sistema operacional. Entretanto, é recomendado utilizar a plataforma Anaconda, que já possui suporte a diferentes ambientes de configuração e oferece interface gráfica de controle de ambientes e instalação de pacotes (você pode optar por utilizar o virtualenv também).

Prossiga com a **instalação do Anaconda** e **execute seu painel** após a instalação. Vá na opção ***environments*** (ambientes) para criar um ambiente específico do Python a fim de rodar o RASA, que será chamado de "*rasa-chatbots*", conforme a Figura 1, apresentada a seguir. Esta etapa é recomendada para evitar conflitos de dependências do RASA com outras bibliotecas, assim você garante o funcionamento ideal sempre.

Criação de um novo ambiente de execução Python, exclusivo para o RASA.

1. Clicar em ***environments***.
2. Clicar em ***create***.
3. Escrever o nome do ambiente (*rasa-chatbots*).
4. Escolher a opção Python (3.7 ou superior).
5. Clicar no botão ***create***.

Figura 1 – Criação de um ambiente de execução para o RASA



Fonte: ©Anaconda Navigator (2021).

Após a criação do ambiente, você pode **clicar no ícone de *play*** (►) para abrir um *prompt* do ambiente criado. A partir deste momento, todas as execuções que realizarmos afetarão apenas este ambiente.

Agora, é preciso de fato instalar o pacote RASA em nosso ambiente, o que é feito ao executar o comando:

pip install rasa

Uma das grandes vantagens de utilizar o RASA é que ele já implementa um *pipeline* de PLN para manipulação de textos; dentro dos itens inclusos, estão os modelos de linguagem da biblioteca spaCy, que, por sua vez, têm suporte ao idioma português. Visto que é de nosso interesse utilizar tais modelos em nosso *pipeline*, é necessária também a instalação da biblioteca spaCy e de seu modelo em português, conforme os comandos a seguir:

pip install spacy

Após a instalação do spaCy, é possível fazer o *download* do modelo em português. Você pode optar pelo modelo pequeno (_sm), médio (_md) ou grande (_lg), basta modificar a última parte do comando para definir qual deles você deseja:

python –m spacy download pt_core_news_md

Pronto, agora já é possível fazer o uso do RASA em nosso ambiente!

Iniciando um projeto

Como o RASA segue uma série de **padrões para nomenclatura de arquivos e hierarquia de diretórios**, recomenda-se a criação de um projeto padrão inicial, que já contempla os arquivos e diretórios principais com alguns exemplos simples em inglês.

Usando o mesmo *prompt* de comando aberto na etapa de instalação, prossiga até o diretório que deseja colocar o projeto, e então execute o comando a seguir.

python –m rasa init

Quando perguntado pelo diretório de inicialização, basta pressionar *enter*. E quando perguntar se já deseja treinar um modelo inicial, pode informar *yes* ou *no* (este processo pode demorar alguns minutos). Ao final, você será perguntado se deseja testar seu agente (*yes* ou *no*). Caso opte por não iniciar agora, você sempre pode fazê-lo com o comando a seguir:

python –m rasa shell

Para verificar se tudo ocorreu corretamente, você deve identificar no diretório que acessou, os arquivos padrão do RASA, como o *config.yml*, *data/nlu.yml* e *data/stories.yml*.

Uma vez que você efetue alterações nos arquivos do seu projeto, você pode treinar seu modelo utilizando o comando a seguir.

python –m rasa train

Todos outros comandos da biblioteca estão descritos aqui.

Entendendo os conceitos do RASA

Com o projeto padrão iniciado, vamos agora entender como funcionam os principais módulos da arquitetura do RASA, saber como customizar nosso agente, e como funcionam os arquivos gerados na pasta do projeto.

Configurações (config.yml): este arquivo define os componentes utilizados pelo agente, bem como as políticas que o modelo utilizará para fazer as predições. Uma descrição detalhada das opções está descrita na documentação. Por enquanto, importa apenas mudar o idioma para português e definir um *pipeline* que utilize o modelo do spaCy em português, conforme o exemplo a seguir.

```
# Configuration for Rasa NLU.
  # https://rasa.com/docs/rasa/nlu/components/
```

```
language: pt

pipeline:
  - name: SpacyNLP
    model: "pt_core_news_md"
  - name: SpacyTokenizer
  - name: SpacyFeaturizer
  - name: RegexFeaturizer
  - name: LexicalSyntacticFeaturizer
  - name: CountVectorsFeaturizer
  - name: CountVectorsFeaturizer
    analyzer: "char_wb"
    min_ngram: 1
    max_ngram: 4
  - name: DIETClassifier
    epochs: 100
  - name: EntitySynonymMapper
  - name: ResponseSelector
    epochs: 100
```

NLU (nlu.yml): o objetivo do *Natural Language Understanding* (NLU) é extrair informações estruturadas das mensagens do usuário. Isso geralmente inclui a intenção (*intent*) do usuário e quaisquer entidades (*entities*) que sua mensagem contenha.

O módulo de NLU do RASA já oferece uma lista de intenções e palavras-chave padrão em inglês, que se traduzidos, podem ser aproveitados por seu projeto. Aqui, você deve personalizar este arquivo de acordo com seu projeto. A seguir, um exemplo do arquivo apenas com as intenções de saudação e despedida, com os exemplos fornecidos para treinamento posterior do modelo.

```
version: "2.0"
nlu:
- intent: saudacao
  examples: |
    - Olá
    - Oie
    - Oi
    - E aí
    - Beleza
    - Bom dia
    - Boa tarde
    - Boa noite
- intent: agradecimento
  examples: |
    - Obrigado
    - Muito obrigado
    - Valeu
    - Agradecido
```

***Stories* (stories.yml):** são um tipo de dado de treinamento usado para treinar o modelo de gerenciamento de diálogo de seu assistente. Eles podem ser usados para treinar modelos que são capazes de generalizar para caminhos de conversação não previstos.

Um *story* é uma representação de uma conversa entre um usuário e um agente, convertida em um formato específico em que as entradas do usuário são expressas como intenções (e entidades quando necessário), enquanto as respostas e ações do assistente são expressas como nomes de ação.

O RASA ainda dá a possibilidade de treinar seu modelo utilizando uma interface de conversação interativa, em que você pode ir ensinando o modelo acerca dos caminhos possíveis, conforme demonstrado nesta seção. A seguir, confira alguns exemplos definidos no arquivo correspondente.

```
version: "2.0"
stories:
- story: caminho feliz
  steps:
  - intent: saudacao
  - action: utter_saudacao
  - intent: humor_otimo
  - action: utter_feliz

- story: caminho triste
  steps:
  - intent: saudacao
  - action: utter_saudacao
  - intent: humor_triste
  - action: utter_anime_se
  - action: utter_isso_ajudou
  - intent: affirm
  - action: utter_feliz
```

Rules **(rules.yml)**: as regras descrevem pequenas partes de conversas que devem seguir sempre o mesmo caminho. Não abuse das regras! Elas são ótimas para lidar com pequenos padrões de conversação específicos, mas, ao contrário dos *stories*, não têm o poder de generalizar para caminhos de conversação não previstos. **Combine regras e *stories*** para tornar seu agente robusto e capaz de lidar com o comportamento real do usuário.

```
version: "2.0"
rules:
- rule: Se despede sempre que o usuário diz tchau
  steps:
  - intent: despedida
  - action: utter_despedida

- rule: Fala “Eu sou um robô” toda vez que usuário perguntar
  steps:
  - intent: pergunta_bot
  - action: utter_bot
```

Domain **(domain.yml)**: o arquivo de domínio especifica as intenções, respostas, lacunas, formulários e ações que seu agente deve conhecer. Além de definir configurações para as sessões de conversação. A seguir, veja um exemplo parcial, retirado de um exemplo de agente de pizzaria.

```
version: "2.0"

intents:
  - saudacao
  - pedir_pizza
```

```
    - cardapio
    - tamanhos
    - agradecimento

  entities:
    - tamanho
    - sabor
    - qtd_sabor

  responses:
    utter_saudacao:
    - text: "Seja bem-vindo a Pizzaria do estudante. O que você deseja?"
    utter_cardapio:
    - text: "Servimos pizzas de calabresa, portuguesa e marguerita. Não é possível pedir pizza de dois sabores."
    utter_tamanhos:
    - text: "Temos as pizzas pequena, média e grande."
    utter_confirmar_pedido:
    - text: "Confirmando o pedido: pizza de {sabor} tamanho {tamanho}?"
    utter_tamanho:
    - text: "Qual tamanho de pizza você deseja? "
    utter_sabores:
    - text: "Qual sabor?"
    utter_despedida:
    - text: "Até mais. Obrigado por pedir com a gente."

  slots:
    tamanho:
      initial_value: null
      type: text
    sabor:
      initial_value: null
      type: text
    qtd_sabor:
      initial_value: null
      type: text

  session_config:
    session_expiration_time: 60
    carry_over_slots_to_new_session: true
```

Actions: ações são as coisas que o seu *bot* pode realmente fazer. Por exemplo, uma ação poderia:

- Responder um usuário.
- Fazer uma chamada a uma API externa.
- Consultar um banco de dados.
- Alguma outra ação via código.

Caso você construa uma ação customizada, deve listar esta ação no arquivo de domínio. Além disso, você deve programar esta ação no arquivo *actions*/actions.py. Uma vez definida a sua rotina, você deverá sempre subir um servidor, que será responsável por lidar com as ações do seu agente. Para tal, é necessário executar o seguinte comando:

rasa run actions

Não se esqueça também de colocar uma referência a este servidor de ações no arquivo endpoints.yml, conforme o exemplo a seguir:

action_endpoint:

url: "http://localhost:5055/webhook"

Os demais módulos e arquivos do RASA são descritos detalhadamente em sua documentação.

Melhoria contínua utilizando o RASA

O RASA oferece alguns recursos muitos interessantes para testar e proporcionar melhorias ao nosso modelo de agente conversacional, conforme apresentado a seguir.

RASA NLU

O módulo de NLU do RASA é responsável por identificar as *intents* relacionadas à interação do usuário, ou seja, sabermos qual a intenção do usuário com a sua fala. Por vezes, inserimos diversas frases de treinamento no arquivo nlu.yml, e mesmo assim o algoritmo identifica incorretamente a *intent* relacionada. A fim de testarmos o módulo de NLU em nosso projeto, podemos fazer uso na unidade de teste do NLU. Para tal, basta irmos até o diretório de nosso projeto e efetuarmos a seguinte chamada:

```
rasa shell nlu
```

A partir disso, será carregado o último modelo treinado para seu agente, e a linha de comando aguardará por uma entrada. Então, para cada mensagem informada, o modelo responderá com a **taxa de confiança** do módulo de NLU para identificação da *intent*. Quanto mais alta a taxa de confiança, maior a chance da *intent* ser recuperada.

No exemplo dado na Figura 2, a frase “olá” é informada, e com taxa de confiança de 0.999 (*i.e.*, 99%), a *intent* de saudação é acionada. A taxa de confiança das demais *intents* também é mostrada. Desta maneira, é possível testarmos de maneira rápida o módulo de NLU de nosso agente, e sabermos se são necessários novos exemplos de treinamento, ou se temos *intents* ambíguas que necessitam de contextualização para funcionarem melhor.

Figura 2: Execução do RASA NLU - *intents*

```
Administrador: C:\Windows\system32\cmd.exe - rasa shell nlu
NLU model loaded. Type a message and press enter to parse it.
Next message:
Olá
{
  "text": "Olá",
  "intent": {
    "id": -2464138808437628533,
    "name": "saudacao",
    "confidence": 0.9999641180038452
  },
  "entities": [],
  "intent_ranking": [
    {
      "id": -2464138808437628533,
      "name": "saudacao",
      "confidence": 0.9999641180038452
    },
    {
      "id": 7865993608337109534,
      "name": "informa_pedido",
      "confidence": 1.5465833712369204e-05
    },
    {
      "id": 341814897514486674,
      "name": "cancela_pedido",
      "confidence": 7.345181074924767e-06
    },
    {
      "id": 5536100204647417066,
      "name": "bot_challenge",
      "confidence": 6.720470992149785e-06
    },
    {
      "id": -5997891465893554440,
      "name": "despedida",
      "confidence": 5.206765308685135e-06
    },
    {
      "id": 3860599691942237594,
      "name": "confirma_pedido",
      "confidence": 8.04234900897427e-07
    },
    {
      "id": -5426094387315097546,
      "name": "inicio_pedido",
      "confidence": 3.162087409691594e-07
    }
  ],
```

Exemplo de execução do comando RASA *shell nlu* identificando as *intents* relacionadas. Fonte: Autor (2021).

Além das *intents*, o módulo de NLU é responsável também por **identificar as entidades** presentes no texto, desta maneira, é possível também visualizar taxas de confiança relacionadas às entidades recuperadas pelo algoritmo, conforme exemplo da Figura 3. Para o texto “quero uma *pizza* grande”, na seção “*entities*”, podemos ver que o extrator “DIETClassifier” (presente no *pipeline* definido em config.yml) identificou com ~88% de confiança o texto “grande” como a entidade “tamanho”.

Figura 3: Execução do RASA NLU: Entidades

#ParaTodosVerem

```
Administrador: C:\Windows\system32\cmd.exe - rasa shell nlu
Next message:
Quero uma pizza grande
{
  "text": "Quero uma pizza grande",
  "intent": {
    "id": 7865993608337109534,
    "name": "informa_pedido",
    "confidence": 0.992373526096344
  },
  "entities": [
    {
      "entity": "tamanho",
      "start": 16,
      "end": 22,
      "confidence_entity": 0.8796650171279907,
      "value": "grande",
      "extractor": "DIETClassifier"
    }
  ],
  "intent_ranking": [
    {
      "id": 7865993608337109534,
      "name": "informa_pedido",
      "confidence": 0.992373526096344
    },
    {
      "id": -5426094387315097546,
      "name": "inicio_pedido",
      "confidence": 0.006909210700541735
    },
```

Exemplo de execução do comando RASA *shell nlu* identificando as *intents* e entidades relacionadas. Fonte: Autor (2021).

RASA interactive

É recomendável que a equipe de desenvolvimento de um agente conversacional realize testes intensivos antes de liberar seu agente em produção. Com o comando ***rasa interactive***, podemos tornar este processo ainda mais produtivo, pois ao mesmo tempo que validamos nosso agente, estaremos também melhorando seu treinamento, ao: (i) confirmar a identificação correta de *intents*, entidades e actions; (ii) fornecer novos exemplos de treinamento para o módulo de NLU; (iii) descrever novos *stories*.

Ao utilizar o RASA *interactive*, a cada nova interação com o usuário, o RASA irá confirmar com o usuário se as classificações foram feitas de maneira correta, conforme a Figura 4, apresentada a seguir. Além disso, ele também confirmará as interações e ações que serão acionadas, bem como mostrará um histórico de interações, que mais tarde servirão para definir novos *stories* em seu arquivo stories.yml, caso o caminho percorrido ainda não tenha sido estabelecido no arquivo.

Caso o módulo de NLU tenha alguma dificuldade em interpretar a entrada do usuário, é possível estabelecer novas *intents*, ou relacionar a entrada à uma *intent* já existente, assim o próprio RASA já efetua atualizações em seus exemplos de treinamento, de maneira automática.

Ao final da execução, o RASA pergunta se o usuário deseja atualizar os arquivos de acordo com os novos exemplos de treinamento vistos durante a utilização do *rasa interactive*. Veja que esta é uma funcionalidade excelente, até mesmo quando você ainda não tem noção do fluxo conversacional de seu agente, pois todo esse fluxo pode ser gerado dinamicamente a partir do uso desta ferramenta.

Figura 4: Execução do RASA *interactive* – Confirmação de identificação de *intent*

```
Bot loaded. Visualisation at http://localhost:5006/visualization.html .
Type a message and press enter (press 'Ctrl-c' to exit).
? Your input -> Gostaria de fazer um pedido
? Your NLU model classified 'Gostaria de fazer um pedido' with intent 'inicio_p
edido' and there are no entities, is this correct? Yes
------
Chat History

 #    Bot                                   You
 1    action_listen
 2                         Gostaria de fazer um pedido
                            intent: inicio_pedido 1.00


Current slots:
        tamanho: None, session_started_metadata: None

------
? The bot wants to run 'utter_inicio_pedido', correct? Yes
------
Chat History

 #    Bot                                                       You
 1    action_listen
 2                                            Gostaria de fazer um pedido
                                               intent: inicio_pedido 1.00
 3    utter_inicio_pedido 1.00
      Servimos apenas o sabor de
      calabresa nos tamanhos pequeno,
      médio e grande. Nos valores 15, 25
      e 30 respectivamente. Qual você
      deseja?


Current slots:
        tamanho: None, session_started_metadata: None

------
? The bot wants to run 'action_listen', correct? Yes
? Your input ->
```

Exemplo de execução do comando Rasa *interactive*, em que é enviada ao usuário uma confirmação de identificação correta da *intent*. Fonte: Autor (2021).

RASA X

Da mesma forma que o RASA *interactive*, você pode oferecer um *link* de acesso ao seu agente para diversos usuários, e utilizar suas interações para melhorar o desempenho dele. Para isso, podemos acionar o RASA X, que utiliza o conceito de *Conversation-Driven Development* (CDD), fazendo uso das interações dos usuários para moldar o agente, visando a capturar comportamentos não esperados no início do desenvolvimento e tornar o agente muito mais robusto.

Ou seja, o RASA X oferece uma interface conversacional de seu agente com usuário, e permite que a equipe de desenvolvimento revise este histórico de conversas a fim de anotar *intents* e entidades definidas no texto e, com isso, melhorar o desempenho do agente, tanto na classificação de *intents* quanto no fluxo conversacional.

O RASA X é um projeto gratuito, paralelo ao RASA, que deve ser instalado para sua utilização:

```
pip install rasa-x --extra-index-url https://pypi.rasa.com/simple
```

Após a instalação, você deve ir até o diretório de seu projeto e acionar o RASA X:

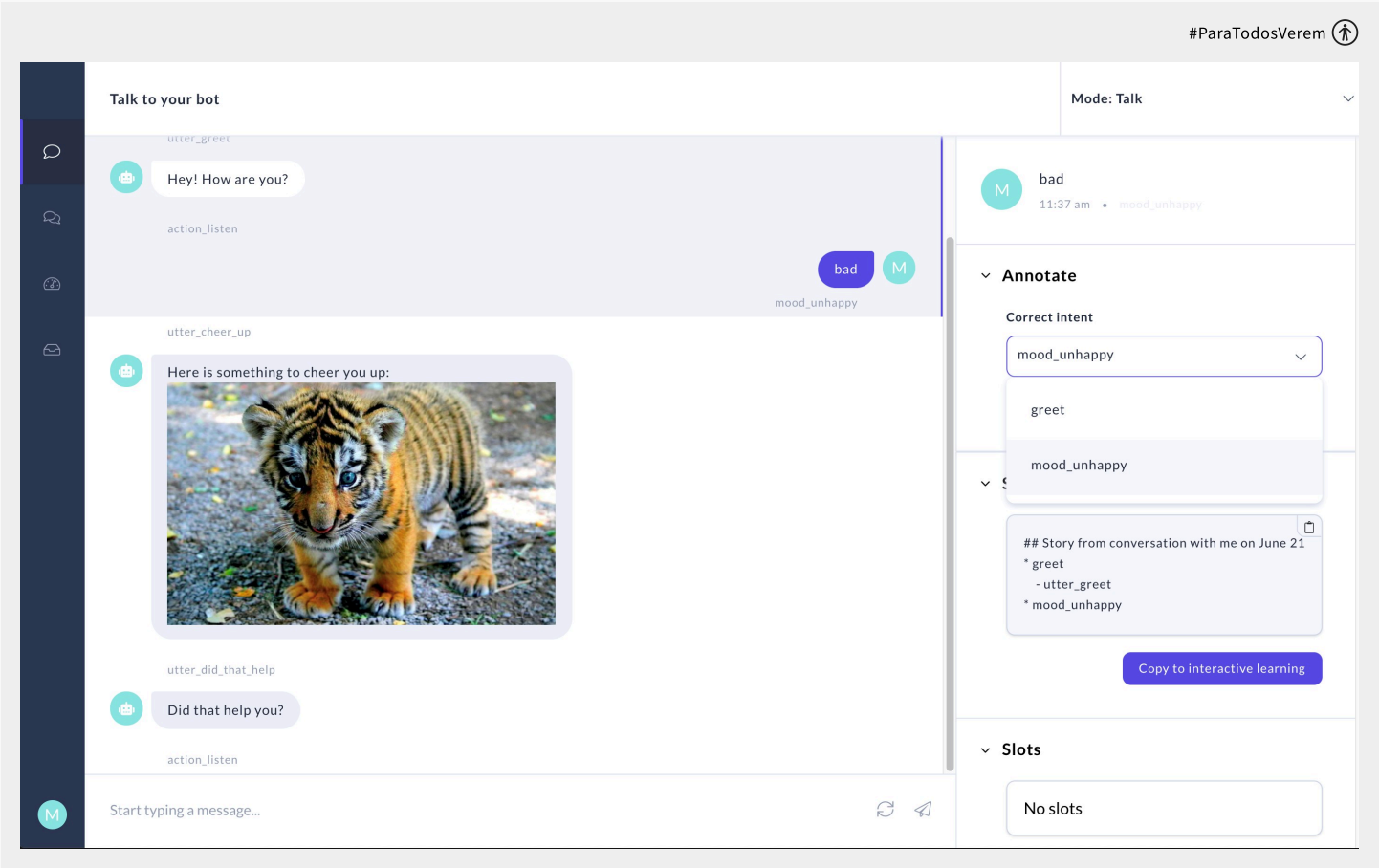
```
rasa x
```

Você pode consultar a documentação para saber mais detalhes sobre como configurar dados de acesso e liberar seu servidor local do RASA X para acesso externo, caso necessário.

Uma vez que você tenha um servidor do RASA X rodando, você pode compartilhar um *link* de acesso com os usuários de teste, que vão ajudá-lo no processo de desenvolvimento, uma vez que podem orientar melhor o comportamento dos usuários, ao fornecer exemplos reais de interação. O quanto antes você liberar acesso do protótipo aos usuários, melhor, e quanto mais melhorias você for fazendo, deve liberar para mais usuários, e pedir que os usuários que já testaram o utilizem novamente.

Uma interface de interação com o seu agente será mostrada ao usuário (conforme a Figura 5, apresentada a seguir), e funcionalidades do modo interativo serão fornecidas, em que é possível realizar correções no funcionamento do agente no que diz respeito aos *intents*, entidades, *slots*, ações e *stories*.

Figura 5: Execução do RASA X



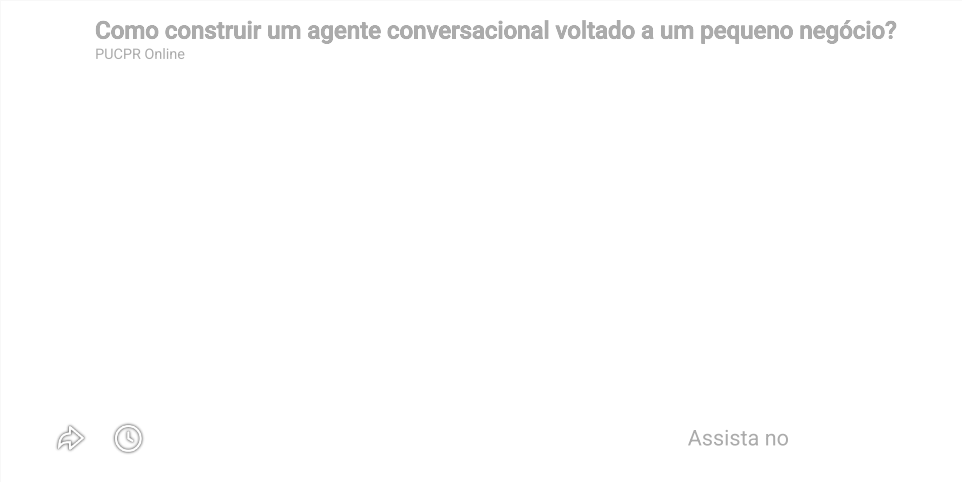
Exemplo de execução do RASA X no *template*-padrão construído pelo RASA. Fonte: Autor (2021).

Uma vez que é formada uma base de dados de interações entre o agente e os usuários de teste, o RASA X oferece a possibilidade de (i) revisar as conversações; (ii) anotar exemplos relevantes para o módulo de NLU; (iii) testar seu agente; (iv) monitorar o progresso em suas melhorias; (v) corrigir os problemas encontrados.

Mãos na massa

Agora que você foi apresentado a uma visão geral do RASA, montaremos juntos um agente conversacional para atender clientes de uma pizzaria. A ideia não é construir um agente completo e com funcionamento perfeito, mas apenas mostrar como você pode iniciar um projeto do RASA e ir customizando seus arquivos de configuração.

Neste vídeo, você poderá acompanhar o desenvolvimento de agente conversacional voltado para o atendimento de clientes de uma pizzaria, utilizando o *framework* RASA.



DOCUMENTOS
Código-fonte

Este é o link do repositório que contém o código-fonte desenvolvido no vídeo.

https://github.com/kunkaweb/rasa-pizza-chatbot/

Fonte: Autor (2021).

Aqui termina a Unidade de Estudo, na qual foram abordadas as seguintes temáticas:

- Uma **visão geral** sobre o *framework* RASA.
- Os **principais conceitos** do RASA ao entender os objetivos de cada arquivo.
- A **construção de um agente** utilizando o *framework*.

Referências

MANTHA, M. Conversational AI for developers: the four essential layers in your stack. **Towards Data Science**, 2020. Disponível em: https://towardsdatascience.com/conversational-ai-for-developers-the-four-essential-layers-in-your-stack-77582e47f5e2. Acesso em: 15 dez. 2020.

NORONHA, O. Rasa 2.0: Criando um chatbot com uma poderosa ferramenta Open Source. **RapaduraTech**, 2020. Disponível em: https://medium.com/rapaduratech/rasa-2-0-criando-um-chatbot-com-uma-poderosa-ferramenta-open-source-289430afcfb5. Acesso em: 15 dez. 2020.

RASA. Rasa: developer documentation. **RASA DOCS**, 2020. Disponível em: https://rasa.com/docs/. Acesso em: 15 dez. 2020.

© PUCPR - Todos os direitos reservados.